Augmented Coding & Design

The Genie Eats The Seed Corn

Kent Beck

3. Mai 2025

- **Quelle:** *Software Design: Tidy First?* (Substack-Newsletter)
- **URL:** https://newsletter.kentbeck.com/p/augmented-coding-and-design
- **Veröffentlicht:** 3. Mai 2025
- **Abgerufen:** 21. August 2026

Hinweis: Das Original enthält mehrere schematische Diagramme (Feature-/Options-Achsen, „inhibiting loop"), die hier nicht wiedergegeben sind. Der Text folgt dem Original wörtlich, einschließlich Becks Gebrauch des Ampersands anstelle von „and".

Farmers in olden times had a saying, "Don't eat the seed corn." Better to go hungry in the spring, plant the corn, & eat later. My coding genie, unfortunately, doesn't know this saying.

Features & Options

In Empirical Software Design we separate progress into two dimensions:

- Features
- Options

To implement a feature you write a new test & make it run. Along the way we may degrade the design by increasing coupling or reduce cohesion. To increase the optionality of the software we refine the structure, reducing coupling or increasing cohesion.

Let's say we've followed this process for a while & now we have a system with some features & some options:

[Diagramm im Original]

Breathing

The empirical style is to add the next feature, then improve the structure.

Then we keep doing that.

This feels like breathing to me. My lungs expand as I take in complexity then relax as I partition that complexity through better design. (There are endless arguments about the time scale, maintain vision, etc. Another day.)

Genie Just Inhales

One of the magical things about augmented coding is how the genie goes past my stated requirements to implement what I would have wanted had I thought to ask for it. "Oh, & here's a command line tool for this." "Oh, & I implemented a special purpose testing tool for that."

One of the not-so-magical things about today's genies is their lack of taste. That giant function? I just added another 20 lines to it. That direct field access? I just used it 20 more times.

The genie seems to assume that its planetary-sized brain is capable of handling any amount of complexity, so it needn't ever reduce complexity. It's right until it isn't.

It's a combination of the two factors above that I'm battling right now:

- More features introduces more complexity
- More complexity slows development of more features

The classic inhibiting loop:

[Diagramm im Original]

Eventually the friction created exceeds the genie's capacity:

[Diagramm im Original]

We are out of options. The genie spins for hours without being able to correctly implement the next feature. We have to either:

- Start over (I've done this several times but I'm frustrated to abandon promising work)
- Manually improve the design (which I enjoy doing but I'm frustrated that my genie got me into a mess it won't clean up)

Manually improving the design reduces complexity, for me & the genie, & enables developing more features. (Eventually entropy catches up, but that's inevitable.)

Need To Know

I'm experimenting with another approach---only telling the genie what it needs to know for the next step. I don't give it overall context---we aren't implementing a database, we are storing keys & values serialized onto fixed size pages of bytes.

The genie doesn't seem to run ahead with unsustainable feature development by restricting its context. Because the complexity doesn't have time to compound, the genie remains capable of helping me refactor (although I wish it knew about automated refactorings).

That's as far as I've gotten. We are all figuring out the shape & limits of these new tools. But we know we can't eat our seed corn.